Teoria do Modelo Multimodelo

Seminário de BDs NoSQL · Grupo 5 · Parte 1: Teoria do modelo

1. Os 4 modelos NoSQL

Cada modelo NoSQL é especialista em um tipo de dado:

- **Documento:** dados em JSON com esquema flexível (ex.: MongoDB)
- **Chave-valor:** chave → valor, com leitura e escrita muito rápidas, usado para cache e sessões (ex.: Redis)
- **Grafo:** nós e arestas, focado em relacionamentos (ex.: Neo4j)
- **Colunas:** dados divididos em partições, feito para grande volume de escrita (ex.: Cassandra)

2. O problema: persistência poliglota

Persistência poliglota é usar um banco diferente para cada tipo de dado, termo popularizado por Martin Fowler. Numa rede social, por exemplo, os perfis ficariam no MongoDB, as amizades no Neo4j e as sessões no Redis.

A ideia é boa, mas o custo operacional é alto:

- vários servidores para manter, com backup, segurança e monitoramento de cada um
- uma linguagem de consulta diferente para cada banco
- dados duplicados e a necessidade de manter tudo sincronizado
- consultas que cruzam modelos precisam ser montadas no código da aplicação (sessão no Redis → perfil no Mongo → amigos no Neo4j)

3. A solução: banco multimodelo

É um único motor que suporta vários modelos de dados de forma nativa, com uma linguagem de consulta e uma infraestrutura só. Funciona como um celular, que junta câmera, GPS e música num aparelho só.

Exemplos:

- **ArangoDB:** documento, grafo e chave-valor, consultados com AQL (multimodelo nativo)
- **Azure Cosmos DB:** documento, chave-valor, grafo e colunas, por meio de APIs diferentes
- **Couchbase:** documento e chave-valor
- **PostgreSQL com JSONB:** banco relacional que ganhou suporte a documentos

No nosso projeto, tudo fica no ArangoDB: `usuarios` (documento), `amizades` (grafo) e `sessoes` (chave-valor).

4. Conclusão: quando usar

- **Vale a pena** quando a aplicação tem dados de formatos diferentes que se relacionam muito (rede social, recomendação, detecção de fraude), quando o time é pequeno ou quando se quer uma infraestrutura simples.
- **Não vale a pena** quando é preciso o máximo de desempenho num modelo só (cache pesado vai melhor no Redis, séries temporais gigantes no Cassandra), quando a aplicação usa um modelo só ou quando é importante ter uma comunidade grande.

Frase final: *o multimodelo não substitui os bancos especialistas. Ele simplifica quando a aplicação precisa de vários modelos ao mesmo tempo.*